Week-6: Triggers
Raghav Kuinkel (NP069732)

1. Create a trigger on the Employee table that automatically inserts a record into EmployeeAudit whenever a new employee is added.

QUERY:

Table

```
CREATE TABLE EmployeeAudit
(
  AuditID INT IDENTITY(1,1) PRIMARY KEY,
  EmployeeID INT,
  FullName VARCHAR(50),
  ActionDate DATETIME
);
```

Trigger

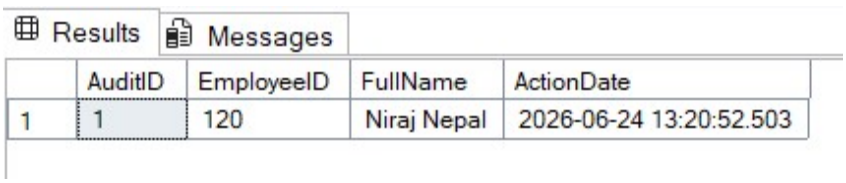
```
CREATE TRIGGER trg_InsertEmployeeAudit
ON Employee
AFTER INSERT
AS
BEGIN
  INSERT INTO EmployeeAudit
  (
    EmployeeID,
    FullName,
    ActionDate
  )
  SELECT
    EmployeeID,
    FullName,
    GETDATE()
  FROM inserted;
END;
```

Test

```
INSERT INTO Employee
VALUES
(120,'Niraj Nepal','IT','Officer',55000,'2024-01-01','Kathmandu');

SELECT * FROM EmployeeAudit;
```

Output:



Results | Messages

	AuditID	EmployeeID	FullName	ActionDate
1	1	120	Niraj Nepal	2026-06-24 13:20:52.503

2. Create a trigger that records the EmployeeID, Employee Name, and current date into an EmployeeLog table whenever a new employee is inserted.

QUERY:

Table

```
CREATE TABLE EmployeeLog
(
  LogID INT IDENTITY(1,1),
  EmployeeID INT,
  EmployeeName VARCHAR(50),
  LogDate DATETIME
);
```

Trigger

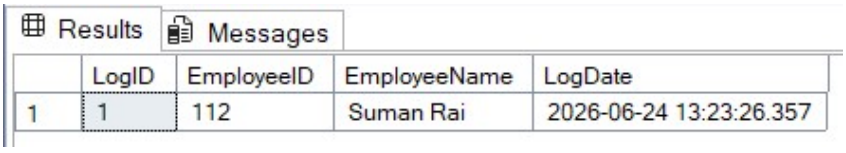
```
CREATE TRIGGER trg_EmployeeLog
ON Employee
AFTER INSERT
AS
BEGIN
  INSERT INTO EmployeeLog
  (
    EmployeeID,
    EmployeeName,
    LogDate
  )
  SELECT
    EmployeeID,
    FullName,
    GETDATE()
  FROM inserted;
END;
```

Test

```
INSERT INTO Employee
VALUES
(112,'Suman Rai','HR','Officer',45000,'2024-01-01','Pokhara');

SELECT * FROM EmployeeLog;
```

OUTPUT



	LogID	EmployeeID	EmployeeName	LogDate
1	1	112	Suman Rai	2026-06-24 13:23:26.357

3. Create a trigger that stores deleted employee details into an EmployeeArchive table whenever an employee record is deleted.

QUERY:

Table

```
CREATE TABLE EmployeeArchive
(
  EmployeeID INT,
  FullName VARCHAR(50),
  Department VARCHAR(30),
  Salary DECIMAL(10,2),
  DeletedDate DATETIME
);
```

Trigger

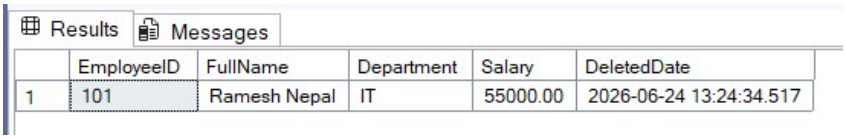
```
CREATE TRIGGER trg_EmployeeArchive
ON Employee
AFTER DELETE
AS
BEGIN
  INSERT INTO EmployeeArchive
  (
    EmployeeID,
    FullName,
    Department,
    Salary,
    DeletedDate
  )
  SELECT
    EmployeeID,
    FullName,
    Department,
    Salary,
    GETDATE()
  FROM deleted;
END;
```

Test

```
DELETE FROM Employee
WHERE EmployeeID = 101;

SELECT * FROM EmployeeArchive;
```

Output:



Results | Messages

	EmployeeID	FullName	Department	Salary	DeletedDate
1	101	Ramesh Nepal	IT	55000.00	2026-06-24 13:24:34.517

4. Create a trigger that records the EmployeeID and deletion date in an EmployeeAudit table whenever an employee is removed.

QUERY:

Table

```
CREATE TABLE EmployeeDeleteAudit
(
  EmployeeID INT,
  DeletionDate DATETIME
);
```

Trigger

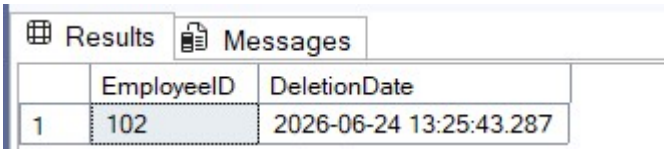
```
CREATE TRIGGER trg_DeleteAudit
ON Employee
AFTER DELETE
AS
BEGIN
  INSERT INTO EmployeeDeleteAudit
  (
    EmployeeID,
    DeletionDate
  )
  SELECT
    EmployeeID,
    GETDATE()
  FROM deleted;
END;
```

Test

```
DELETE FROM Employee
WHERE EmployeeID = 102;

SELECT * FROM EmployeeDeleteAudit;
```

OUTPUT:



5. Create a trigger that records old salary and new salary whenever an employee's salary is updated.

QUERY:

Table

```
CREATE TABLE SalaryHistory
(
  EmployeeID INT,
  OldSalary DECIMAL(10,2),
  NewSalary DECIMAL(10,2),
  ChangeDate DATETIME
);
```

Trigger

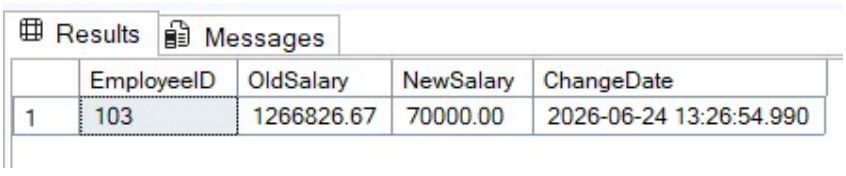
```
CREATE TRIGGER trg_SalaryHistory
ON Employee
AFTER UPDATE
AS
BEGIN
  INSERT INTO SalaryHistory
  (
    EmployeeID,
    OldSalary,
    NewSalary,
    ChangeDate
  )
  SELECT
    d.EmployeeID,
    d.Salary,
    i.Salary,
    GETDATE()
  FROM deleted d
  INNER JOIN inserted i
  ON d.EmployeeID=i.EmployeeID
  WHERE d.Salary<>i.Salary;
END;
```

Test

```
UPDATE Employee
SET Salary=70000
WHERE EmployeeID=103;

SELECT * FROM SalaryHistory;
```

OUTPUT:



Results | Messages

	EmployeeID	OldSalary	NewSalary	ChangeDate
1	103	1266826.67	70000.00	2026-06-24 13:26:54.990

6. Create a trigger that records all department changes made to employees in a DepartmentHistory table.

QUERY:

Table

```
CREATE TABLE DepartmentHistory
(
  EmployeeID INT,
  OldDepartment VARCHAR(30),
  NewDepartment VARCHAR(30),
  ChangeDate DATETIME
);
```

Trigger

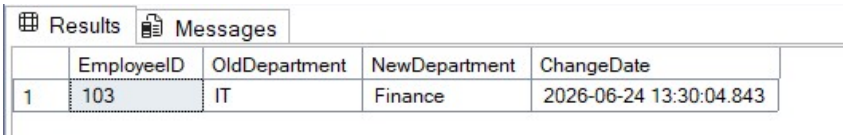
```
CREATE TRIGGER trg_DepartmentHistory
ON Employee
AFTER UPDATE
AS
BEGIN
  INSERT INTO DepartmentHistory
  SELECT
    d.EmployeeID,
    d.Department,
    i.Department,
    GETDATE()
  FROM deleted d
  INNER JOIN inserted i
  ON d.EmployeeID=i.EmployeeID
  WHERE d.Department<>i.Department;
END;
```

Test

```
UPDATE Employee
SET Department='Finance'
WHERE EmployeeID=103;

SELECT * FROM DepartmentHistory;
```

OUTPUT:



Results | Messages

	EmployeeID	OldDepartment	NewDepartment	ChangeDate
1	103	IT	Finance	2026-06-24 13:30:04.843

7. Create a trigger that automatically stores EmployeeID, old designation, and new designation whenever an employee's designation changes.

QUERY:

Table

```
CREATE TABLE DesignationHistory
(
  EmployeeID INT,
  OldDesignation VARCHAR(30),
  NewDesignation VARCHAR(30),
  ChangeDate DATETIME
);
```

Trigger

```
CREATE TRIGGER trg_DesignationHistory
ON Employee
AFTER UPDATE
AS
BEGIN
  INSERT INTO DesignationHistory
  SELECT
    d.EmployeeID,
    d.Designation,
    i.Designation,
    GETDATE()
  FROM deleted d
  INNER JOIN inserted i
  ON d.EmployeeID=i.EmployeeID
  WHERE d.Designation<>i.Designation;
END;

Test
UPDATE Employee
SET Designation='Manager'
WHERE EmployeeID=103;

SELECT * FROM DesignationHistory;
```

OUTPUT:

Results | Messages

	EmployeeID	OldDesignation	NewDesignation	ChangeDate
1	103	Developer	Manager	2026-06-24 13:31:15.070

8. Create a trigger that records every UPDATE operation performed on the Employee table in an Audit table.

QUERY:

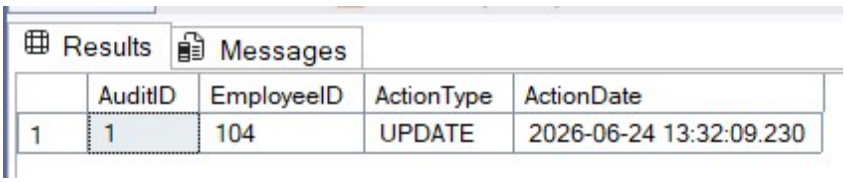
```
Table
CREATE TABLE AuditTable
(
  AuditID INT IDENTITY(1,1),
  EmployeeID INT,
  ActionType VARCHAR(20),
  ActionDate DATETIME
);

Trigger
CREATE TRIGGER trg_UpdateAudit
ON Employee
AFTER UPDATE
AS
BEGIN
  INSERT INTO AuditTable
  (
    EmployeeID,
    ActionType,
    ActionDate
  )
  SELECT
    EmployeeID,
    'UPDATE',
    GETDATE()
  FROM inserted;
END;

Test
UPDATE Employee
SET Salary=80000
WHERE EmployeeID=104;

SELECT * FROM AuditTable;
```

OUTPUT:



	AuditID	EmployeeID	ActionType	ActionDate
1	1	104	UPDATE	2026-06-24 13:32:09.230

9. Create a trigger that records every INSERT, UPDATE, and DELETE operation performed on the Employee table in a TransactionLog table.

QUERY:

Table

```
CREATE TABLE TransactionLog
(
  LogID INT IDENTITY(1,1),
  EmployeeID INT,
  ActionType VARCHAR(20),
  ActionDate DATETIME
);
```

Trigger for INSERT

```
CREATE TRIGGER trg_InsertLog
ON Employee
AFTER INSERT
AS
BEGIN
  INSERT INTO TransactionLog SELECT EmployeeID,'INSERT',GETDATE()
  FROM inserted;
END;
```

Trigger for UPDATE

```
CREATE TRIGGER trg_UpdateLog
ON Employee
AFTER UPDATE
AS
BEGIN
  INSERT INTO TransactionLog SELECT EmployeeID,'UPDATE',GETDATE()
  FROM inserted;
END;
```

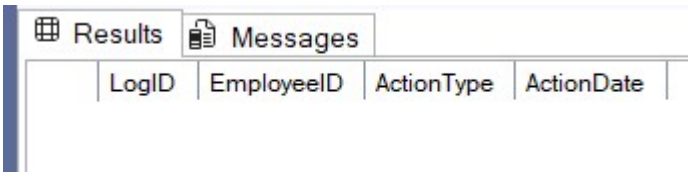
Trigger for DELETE

```
CREATE TRIGGER trg_DeleteLog
ON Employee
AFTER DELETE
AS
BEGIN
  INSERT INTO TransactionLog SELECT EmployeeID,'DELETE',GETDATE()
  FROM deleted;
END;
```

Test

```
SELECT * FROM TransactionLog;
```

OUTPUT:



10. Create a trigger that stores details of employees whose salary exceeds 100000 into a HighSalaryEmployees table after insertion.

QUERY:

Table

```
CREATE TABLE HighSalaryEmployees
(
  EmployeeID INT,
  FullName VARCHAR(50),
  Salary DECIMAL(10,2)
);
```

Trigger

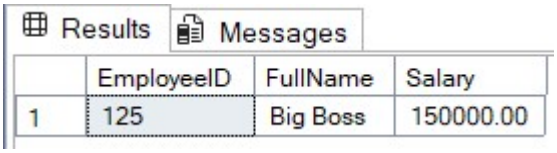
```
CREATE TRIGGER trg_HighSalaryEmployee
ON Employee
AFTER INSERT
AS
BEGIN
  INSERT INTO HighSalaryEmployees
  (
    EmployeeID,
    FullName,
    Salary
  )
  SELECT
    EmployeeID,
    FullName,
    Salary
  FROM inserted
  WHERE Salary > 100000;
END;
```

Test

```
INSERT INTO Employee
VALUES
(125,'Big Boss','IT','Director',150000,'2024-01-01','Kathmandu');

SELECT * FROM HighSalaryEmployees;
```

OUTPUT:



	EmployeeID	FullName	Salary
1	125	Big Boss	150000.00

11. Create a trigger that displays a message whenever an employee is transferred to a different department.

QUERY:
Create Transfer Log Table

```
CREATE TABLE DepartmentTransferLog
(
  EmployeeID INT,
  EmployeeName VARCHAR(50),
  OldDepartment VARCHAR(30),
  NewDepartment VARCHAR(30),
  TransferDate DATETIME
);
```

Trigger

```
CREATE TRIGGER trg_DepartmentTransfer
ON Employee
AFTER UPDATE
AS
BEGIN
  IF UPDATE(Department)
  BEGIN
    INSERT INTO DepartmentTransferLog
    (
      EmployeeID,
      EmployeeName,
      OldDepartment,
      NewDepartment,
      TransferDate
    )
    SELECT
      d.EmployeeID,
      i.FullName,
      d.Department,
      i.Department,
      GETDATE()
    FROM deleted d
    INNER JOIN inserted i
      ON d.EmployeeID = i.EmployeeID
    WHERE d.Department <> i.Department;
    PRINT 'Employee transferred to another department';
  END
END;
GO
```

Test

```
UPDATE Employee
SET Department = 'Marketing'
WHERE EmployeeID = 105;

SELECT * FROM DepartmentTransferLog;
```

OUTPUT: